



Guia tecnico das tecnologias do projeto

Como cada tecnologia aparece na TSZR15, como elas se conectam e quais estruturas de código e banco sustentam a loja.

Campo	Valor
Projeto	tszr15-whatsapp-store
Versão do app	0.1.0
Branch	fix/vazamento-erros-cliente
Commit base	a06a9c20
Gerado em	07/08/2026 14:58
Fonte	Arquivos locais do repositório, package.json e supabase/migrations

Escopo

Este PDF documenta as tecnologias que aparecem no código atual. Integrações listadas em docs/APIS-NECESSARIAS.md entram em uma seção separada como tecnologias planejadas, não como recursos já implementados.

1. Visao geral

A TSZR15 e uma loja de pecas e acessorios para Yamaha R15. O frontend e feito em Next.js e React; o backend fica distribuido entre rotas API, Server Actions, proxy de sessao e funcoes RPC no Supabase/Postgres. O fechamento comercial atual usa WhatsApp Business por link wa.me com mensagem montada pelo backend.

Estrutura principal do repositorio

```
app/  
  page.js, catalogo/page.js, produto/[slug]/page.js  
  api/catalog/route.js, api/checkout/whatsapp/route.js  
  auth/actions.js, admin/actions.js, conta/actions.js  
src/  
  catalog/      dados, validacao e leitura do catalogo  
  components/   componentes React da vitrine, carrinho, formularios e admin  
  checkout/     totais, cupom, pedido, mensagem WhatsApp  
  lib/supabase/ clientes browser/server/service_role  
  admin/        sessao admin, pedidos, catalogo, analytics  
  tracking/     visao publica de rastreo  
supabase/migrations/  
  tabelas, RLS, storage buckets e RPCs do banco  
tests/  
  node:test para catalogo, checkout, rate limit, auth e utilitarios  
scripts/  
  validacao e sincronizacao do catalogo
```

O codigo segue uma separacao por responsabilidade: app/ roteia a experiencia; src/ contem dominio e componentes; supabase/ define persistencia.

Fluxo resumido de compra

```
Cliente abre catalogo  
-> Next.js Server Component busca catalogo publicado no Supabase  
-> React Client Component filtra, escolhe produto e salva carrinho no localStorage  
-> Carrinho envia POST /api/checkout/whatsapp  
-> rota API valida itens contra o catalogo confiavel do servidor  
-> backend calcula total/cupom/frete e chama RPC create_checkout_order  
-> Supabase grava orders, order_items, payments e audit_logs  
-> API retorna whatsappUrl  
-> navegador abre https://wa.me/<numero>?text=<mensagem>
```

Inventario das tecnologias em uso

Tecnologia	Onde fica	Funcao no projeto
Next.js App Router	app/, app/api, middleware.js	Rotas, Server Components, API routes, revalidacao e proxy de sessao.
React 19	src/components	Estado local, eventos, carrosseis, busca, carrinho e formularios.
Supabase SSR/JS	src/lib/supabase	Auth, queries, RPC, storage e service_role no servidor.
Postgres + RLS + PL/pgSQL	supabase/migrations	Modelo relacional, politicas de acesso, triggers e funcoes transacionais.
WhatsApp wa.me	src/checkout/whatsapp.js	Gera URL de atendimento com mensagem codificada.
ViaCEP	src/customer/cep-lookup.js	Preenche endereco a partir do CEP.
Pino	src/lib/logger.js	Logs estruturados com redacao de dados sensiveis.
Vercel Speed Insights	app/layout.js	Metrica de performance no layout raiz.
Node test runner	tests/*.mjs	Testes sem framework extra usando node:test.
ESLint + Prettier	eslint.config.mjs, package.json	Padrao de qualidade e formatacao.
GitHub Actions + Gitleaks	.github/workflows	Varredura de secrets em push/PR.

Dependencias declaradas

Pacote	Versao em package.json	Tipo
@supabase/ssr	^0.10.3	runtime
@supabase/supabase-js	^2.106.0	runtime
@vercel/analytics	^2.0.1	runtime
@vercel/speed-insights	^2.0.0	runtime
next	^16.2.4	runtime
react	^19.2.5	runtime
react-dom	^19.2.5	runtime
resend	^6.14.0	runtime
@eslint/js	^10.0.1	dev/tooling
@opennextjs/cloudflare	^1.20.1	dev/tooling
@testing-library/react	^16.3.2	dev/tooling
@testing-library/user-event	^14.6.1	dev/tooling
@types/node	^26.0.0	dev/tooling
@types/react	^19.2.17	dev/tooling
@types/react-dom	^19.2.3	dev/tooling
eslint	^10.4.1	dev/tooling
eslint-config-prettier	^10.1.8	dev/tooling
globals	^17.6.0	dev/tooling
jsdom	^29.1.1	dev/tooling
prettier	^3.8.3	dev/tooling
sharp	^0.35.1	dev/tooling
supabase	^2.100.1	dev/tooling
typescript	^6.0.3	dev/tooling
vitest	^4.1.9	dev/tooling
wrangler	^4.107.1	dev/tooling

As versoes acima vem de package.json, que e a fonte de verdade do projeto. O diretorio node_modules local pode estar inconsistente em maquinas de desenvolvimento e nao deve ser usado como inventario tecnico.

2. Next.js App Router

Next.js organiza a aplicação por arquivos dentro de `app/`. Cada pasta vira uma rota. Arquivos `page.js` renderizam páginas; `route.js` implementam endpoints; `loading.js` define estado de carregamento; `layout.js` envolve todas as páginas.

`app/layout.js` - layout raiz e Speed Insights

```
1 | import "../globals.css";
2 |
3 | import { AuthHashBridge } from "@src/auth/auth-hash-bridge.js";
4 | import { NavigationLoadingOverlay } from
  | "@src/components/loading/navigation-loading-overlay.js";
5 | import { MobileTabBar } from "@src/components/mobile-tab-bar.js";
6 | import { SiteFooter } from "@src/components/site-footer.js";
7 |
8 | export const metadata = {
9 |   metadataBase: new URL("https://www.tszr15-store.com.br"),
10 |   title: "TSZR15 | Loja R15 com conta de cliente",
11 |   description:
12 |     "Catálogo Yamaha R15 com busca, conta de cliente, dados de entrega e fechamento via
  | WhatsApp Business.",
13 |   icons: {
14 |     icon: "/brand/logo-tszr15-store.png"
15 |   }
16 | };
17 |
18 | export default function RootLayout({ children }) {
19 |   return (
20 |     <html data-scroll-behavior="smooth" lang="pt-BR">
21 |       <body>
22 |         <AuthHashBridge />
23 |         <NavigationLoadingOverlay />
24 |         {children}
```

O layout raiz configura idioma, metadados, bridge de autenticação, overlay de navegação e o componente SpeedInsights da Vercel. Isso faz com que toda rota compartilhe a mesma base visual e de comportamento.

`app/page.js` - Server Component com revalidação

```
1 | import globalStyles from "@app/storefront.module.css";
2 | import { cx } from "@src/lib/classnames";
3 | import { CatalogHub } from "@src/components/catalog/catalog-hub.js";
4 | import { getStorefrontMenu } from "@src/catalog/index.js";
5 | import {
6 |   getStorefrontCatalogPage,
7 |   getStorefrontFeaturedProducts
8 | } from "@src/catalog/storefront-query.js";
9 | import { getCachedPublicCatalogProductsForStorefront } from
  | "@src/catalog/supabase-catalog.js";
10 |
11 | // As ações administrativas invalidam estas rotas; o TTL de uma hora cobre alterações
  | externas.
12 | export const revalidate = 3600;
13 |
14 | export default async function HomePage({ searchParams }) {
15 |   const params = await searchParams;
```

A home é async, roda no servidor e usa `revalidate = 3600` para cache incremental. A página busca produtos publicados, monta o menu e entrega os dados prontos ao componente cliente `CatalogHub`.

app/api/catalog/route.js - API route estatica

```
1 | import { getStorefrontMenu } from "@/src/catalog/index.js";
2 | import { getPublicCatalogProductsForStorefront } from
  |   "@/src/catalog/supabase-catalog.js";
3 |
4 | export const dynamic = "force-static";
5 | export const revalidate = 3600;
6 |
7 | export async function GET() {
8 |   const catalog = await getPublicCatalogProductsForStorefront();
9 |
10 |   return Response.json({
11 |     categories: getStorefrontMenu(catalog.products),
12 |     products: catalog.products,
13 |     source: catalog.source
14 |   });
15 | }
```

API routes do App Router exportam funcoes HTTP como GET e POST. Aqui o catalogo publico vira JSON, com `dynamic = force-static` e `revalidate = 3600` para evitar consulta desnecessaria a cada request.

Rotas importantes

Rota	Arquivo	Papel
/	app/page.js	Vitrine principal com catalogo e busca.
/catalogo	app/catalogo/page.js	Entrada dedicada para produtos.
/produto/[slug]	app/produto/[slug]/page.js	Detalhe do produto, imagens, avaliacoes e relacionados.
/pedido	app/pedido/page.js	Carrinho e checkout.
/conta	app/conta/page.js + actions.js	Perfil, enderecos, pedidos e avaliacoes.
/admin/*	app/admin/<secao>/page.js + actions.js	Operacao interna em quatro segmentos: pedidos, produtos, cupons e analise.
/rastreo	app/rastreo/page.js	Consulta publica por pedido e contato.
/api/checkout/whatsapp	app/api/checkout/whatsapp/route.js	Valida e persiste pedido, devolve URL do WhatsApp.

3. React no cliente

Componentes que precisam de estado, eventos do navegador, localStorage ou chamadas fetch no browser usam a diretiva 'use client'. O servidor entrega dados iniciais; React controla interacao fina da vitrine e do checkout.

src/components/catalog/catalog-hub.js - busca, categoria ativa e rendering

```
78 | export function CatalogHub({
79 |   catalogPage,
80 |   catalogTotal,
81 |   categories,
82 |   currentUser,
83 |   featuredProducts = [],
84 |   filters
85 | }) {
86 |   return (
87 |     <>
88 |       <StoreHeader currentUser={currentUser} showSearch={false} />
89 |
90 |       <section className={cx(globalStyles, "brand-hero")}>
91 |         <div className={cx(globalStyles, "brand-hero-copy")}>
92 |           <div className={cx(globalStyles, "hero-brand-row")}>
93 |             <Image
94 |               alt="TSZ Store"
95 |               className={cx(globalStyles, "hero-logo")}
96 |               height={2000}
97 |               sizes="188px"
98 |               src={brandLogoSrc}
99 |               width={2000}
100 |             />
101 |             <p className={cx(globalStyles, "hero-kicker")}>Performance parts for Yamaha
102 |           </p>
103 |           <h1>
104 |             Sua R15 <span>em outro nível</span>
105 |           </h1>
106 |           <p className={cx(globalStyles, "hero-lead")}>
107 |             Peças e acessórios selecionados para quem exige visual agressivo,
108 |             acabamento premium e
109 |             atendimento direto no WhatsApp.
110 |           </p>
111 |           <div className={cx(globalStyles, "hero-actions")}>
112 |             <a className={cx(globalStyles, "button button-primary")} href="#produtos">
113 |               Ver produtos
114 |             </a>
115 |             <a className={cx(globalStyles, "button button-ghost")} href="#lançamentos">
116 |               Conferir lançamentos
117 |             </a>
118 |           </div>
119 |
120 |           <div className={cx(globalStyles, "brand-hero-media")}>
121 |             <div className={cx(globalStyles, "hero-media-frame")}>
122 |               <Image
```

useDeferredValue reduz pressão da busca; startTransition evita travar a UI ao trocar filtros.

src/components/catalog/cart-checkout.js - estados do carrinho

```
// marcador não encontrado em src/components/catalog/cart-checkout.js: const [cartItems
```

O checkout controla carrinho, cupom, frete, consentimento, CEP e status de envio no estado React.

src/components/catalog/catalog-shared.js - localStorage do carrinho

```
// marcador não encontrado em src/components/catalog/catalog-shared.js: export function
readStoredCart
```

O carrinho fica no browser e dispara um evento customizado para atualizar badges sem recarregar a página.

Regra prática

Se o código usa cookies(), headers(), Supabase service_role ou acesso sensível, ele fica no servidor. Se usa localStorage, window, eventos e inputs interativos, ele fica em componente cliente.

4. Supabase: Auth, dados e storage

Supabase aparece como a plataforma de banco, autenticação, storage e RPC. O projeto separa três clientes: browser, server SSR com cookies e `service_role` para operações administrativas ou de convidado no servidor.

src/lib/supabase/config.js - leitura de envs e preview

```
1 | const SUPABASE_URL_ENV_KEYS = ["NEXT_PUBLIC_SUPABASE_URL", "SUPABASE_URL"];
2 | const SUPABASE_PREVIEW_URL_ENV_KEYS = ["NEXT_PUBLIC_SUPABASE_PREVIEW_URL",
  | "SUPABASE_PREVIEW_URL"];
3 | const SUPABASE_PUBLISHABLE_KEY_ENV_KEYS = [
4 |   "NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY",
5 |   "SUPABASE_PUBLISHABLE_KEY",
6 |   "NEXT_PUBLIC_SUPABASE_ANON_KEY",
7 |   "SUPABASE_ANON_KEY"
8 | ];
9 | const SUPABASE_PREVIEW_PUBLISHABLE_KEY_ENV_KEYS = [
10 |   "NEXT_PUBLIC_SUPABASE_PREVIEW_PUBLISHABLE_KEY",
11 |   "SUPABASE_PREVIEW_PUBLISHABLE_KEY",
12 |   "NEXT_PUBLIC_SUPABASE_PREVIEW_ANON_KEY",
13 |   "SUPABASE_PREVIEW_ANON_KEY"
14 | ];
15 | const SUPABASE_SERVICE_ROLE_KEY_ENV_KEYS = ["SUPABASE_SERVICE_ROLE_KEY",
  | "SUPABASE_SECRET_KEY"];
16 | const SUPABASE_PREVIEW_SERVICE_ROLE_KEY_ENV_KEYS = [
17 |   "SUPABASE_PREVIEW_SERVICE_ROLE_KEY",
18 |   "SUPABASE_PREVIEW_SECRET_KEY"
19 | ];
20 |
21 | function isPreviewRuntime() {
22 |   const explicitTarget = String(
23 |     process.env.SUPABASE_RUNTIME_TARGET ??
  | process.env.NEXT_PUBLIC_SUPABASE_RUNTIME_TARGET ?? ""
24 |   ).toLowerCase();
25 |
26 |   if (explicitTarget) {
27 |     return explicitTarget === "preview";
28 |   }
29 |
30 |   return process.env.VERCEL_ENV === "preview";
31 | }
32 |
33 | function firstConfiguredEnvValue(keys) {
34 |   return keys.map((key) => process.env[key]).find(Boolean) ?? "";
35 | }
36 |
37 | function envKeysForRuntime(previewKeys, fallbackKeys) {
38 |   return isPreviewRuntime() ? [...previewKeys, ...fallbackKeys] : fallbackKeys;
39 | }
40 |
41 | export function getSupabaseUrl() {
42 |   return firstConfiguredEnvValue(
43 |     envKeysForRuntime(SUPABASE_PREVIEW_URL_ENV_KEYS, SUPABASE_URL_ENV_KEYS)
44 |   );
45 | }
46 |
47 | export function getSupabasePublishableKey() {
48 |   return firstConfiguredEnvValue(
49 |     envKeysForRuntime(SUPABASE_PREVIEW_PUBLISHABLE_KEY_ENV_KEYS,
  | SUPABASE_PUBLISHABLE_KEY_ENV_KEYS)
50 |   );
51 | }
52 |
53 | export function getSupabaseServiceRoleKey() {
54 |   return firstConfiguredEnvValue(
```

src/lib/supabase/server.js - cliente SSR com cookies

```
1 | import { cookies } from "next/headers";
2 | import { createServerClient } from "@supabase/ssr";
3 |
4 | import { getPublicSupabaseConfig } from "../config.js";
5 |
6 | export async function createServerSupabaseClient() {
7 |   const config = getPublicSupabaseConfig();
8 |
9 |   if (!config.isConfigured) {
10 |     return null;
11 |   }
12 |
13 |   const cookieStore = await cookies();
14 |
15 |   return createServerClient(config.url, config.publishableKey, {
16 |     cookies: {
17 |       getAll() {
18 |         return cookieStore.getAll();
19 |       },
20 |       setAll(cookiesToSet) {
21 |         try {
22 |           cookiesToSet.forEach(({ name, value, options }) => {
23 |             cookieStore.set(name, value, options);
24 |           });
25 |         } catch {
26 |           // Server Components cannot write cookies. The root proxy refreshes sessions.
27 |         }
28 |       }
29 |     }
30 |   });
31 | }
```

src/lib/supabase/client.js - cliente do navegador

```
1 | import { createBrowserClient } from "@supabase/ssr";
2 |
3 | import { getPublicSupabaseConfig } from "../config.js";
4 |
5 | export function createBrowserSupabaseClient() {
6 |   const config = getPublicSupabaseConfig();
7 |
8 |   if (!config.isConfigured) {
9 |     return null;
10 |   }
11 |
12 |   return createBrowserClient(config.url, config.publishableKey);
13 | }
```

src/lib/supabase/admin.js - service_role somente no servidor

```
1 | import "server-only";
2 |
3 | import { createClient } from "@supabase/supabase-js";
4 |
5 | import { getSupabaseServiceRoleKey, getSupabaseUrl } from "../config.js";
6 |
7 | export function createServiceRoleSupabaseClient() {
8 |   const url = getSupabaseUrl();
9 |   const serviceRoleKey = getSupabaseServiceRoleKey();
10 |
11 |   if (!url || !serviceRoleKey) {
12 |     return null;
13 |   }
14 |
15 |   return createClient(url, serviceRoleKey, {
16 |     auth: {
17 |       autoRefreshToken: false,
18 |       persistSession: false
19 |     }
20 |   });
21 | }
```

O cliente `service_role` importa `'server-only'`. Isso evita que código privilegiado seja empacotado para o navegador. Quando a chave não existe, as funções retornam `null` e o app reduz funcionalidade em vez de expor segredo.

middleware.js - refresh de sessao e protecao de /admin

```
41 | export async function middleware(request) {
42 |   const isAdminRequest = isAdminPath(request.nextUrl.pathname);
43 |
44 |   let response = NextResponse.next({ request });
45 |
46 |   if (isAdminRequest) {
47 |     const adminSessionValue = request.cookies.get(ADMIN_SESSION_COOKIE)?.value ?? "";
48 |     const hasSignedAdminSession = await
isAdminSessionValueValidAtEdge(adminSessionValue);
49 |
50 |     if (!hasSignedAdminSession) {
51 |       return redirectToAdminLogin(request);
52 |     }
53 |
54 |     return addAdminSecurityHeaders(response);
55 |   }
56 |
57 |   const supabaseUrl = getSupabaseUrl();
58 |   const supabaseKey = getSupabasePublishableKey();
59 |
60 |   if (!supabaseUrl || !supabaseKey) {
61 |     return response;
62 |   }
63 |
64 |   const supabase = createServerClient(supabaseUrl, supabaseKey, {
65 |     cookies: {
66 |       getAll() {
67 |         return request.cookies.getAll();
68 |       },
69 |       setAll(cookiesToSet) {
70 |         cookiesToSet.forEach(({ name, value }) => request.cookies.set(name, value));
71 |         response = NextResponse.next({ request });
72 |         cookiesToSet.forEach(({ name, value, options }) => {
73 |           response.cookies.set(name, value, options);
74 |         });
75 |       }
76 |     }
77 |   });
78 |
79 |   await supabase.auth.getUser();
80 |
81 |   return response;
82 | }
83 |
84 | export const config = {
85 |   matcher: [
86 |     "/admin/:path*",
87 |     "/auth/:path*",
```

O proxy atualiza cookies de sessao Supabase fora de /api e aplica headers no-store/noindex para area administrativa.

Banco Postgres, RLS e RPC

Area	Tabelas	Origem
Cliente	public.customer_profiles, public.customer_addresses, public.assisted_purchase_consent	Definido em supabase/migrations
Pedido	public.orders, public.order_items, public.payments, public.audit_logs	Definido em supabase/migrations
Operacao	public.supplier_purchases, public.supplier_tracking_events, public.support_threads	Definido em supabase/migrations
Catalogo	public.catalog_categories, public.catalog_products, public.catalog_product_categories, public.catalog_product_costs, public.catalog_coupons	Definido em supabase/migrations
Avaliacoes	public.order_item_reviews, public.order_review_photos	Definido em supabase/migrations
Seguranca	private.api_rate_limits	Definido em supabase/migrations

catalog_products - tabela de catalogo publicada

```
11 | create table if not exists public.catalog_products (  
12 |     id text primary key,  
13 |     slug text not null unique,  
14 |     name text not null,  
15 |     storefront_category_ids text[] not null default '{}',  
16 |     product_family text not null,  
17 |     bike_model_scope text[] not null default '{}',  
18 |     price_cents integer not null check (price_cents > 0),  
19 |     currency text not null default 'BRL',  
20 |     variations text[] not null default '{}',  
21 |     availability text not null default 'sob-consulta',  
22 |     lead_time_days integer not null default 2 check (lead_time_days >= 0),  
23 |     shipping_class text not null default 'medium',  
24 |     checkout_channel text not null default 'whatsapp-business',  
25 |     internal_purchase_source jsonb not null default '{}':jsonb,  
26 |     notes text not null default '',  
27 |     is_published boolean not null default true,  
28 |     created_at timestampz not null default now(),  
29 |     updated_at timestampz not null default now(),  
30 |     constraint catalog_products_currency_check check (currency = 'BRL'),  
31 |     constraint catalog_products_checkout_channel_check check (checkout_channel =  
    'whatsapp-business'),  
32 |     constraint catalog_products_storefront_categories_check check  
    (array_length(storefront_category_ids, 1) >= 1),  
33 |     constraint catalog_products_variations_check check (array_length(variations, 1) >= 1)  
34 | );  
35 |
```

orders - pedido com status, snapshots e canal de atendimento

```
171 | create table if not exists public.orders (  
172 |     id uuid primary key default gen_random_uuid(),  
173 |     order_number text not null unique default public.generate_order_number(),  
174 |     user_id uuid references auth.users(id) on delete set null,  
175 |     customer_name text not null,  
176 |     customer_email text,  
177 |     customer_whatsapp text,  
178 |     customer_phone text,  
179 |     customer_tax_id text,  
180 |     customer_snapshot jsonb not null default '{}':jsonb,  
181 |     address_snapshot jsonb not null default '{}':jsonb,  
182 |     subtotal_cents integer not null default 0 check (subtotal_cents >= 0),  
183 |     discount_cents integer not null default 0 check (discount_cents >= 0),  
184 |     shipping_cents integer not null default 0 check (shipping_cents >= 0),  
185 |     total_cents integer not null default 0 check (total_cents >= 0),  
186 |     currency text not null default 'BRL',  
187 |     shipping_option_id text not null,  
188 |     shipping_label text not null,  
189 |     shipping_eta text,  
190 |     payment_method_id text not null,  
191 |     payment_method_label text not null,  
192 |     payment_status text not null default 'aguardando_pagamento'  
193 |     check (payment_status in (  
194 |         'aguardando_pagamento',  
195 |         'pagamento_confirmado',  
196 |         'cancelado',  
197 |         'reembolsado'  
198 |     )),  
199 |     operational_status text not null default 'enviado_whatsapp_business'  
200 |     check (operational_status in (  
201 |         'orcamento_iniciado',  
202 |         'enviado_whatsapp_business',  
203 |         'aguardando_atendimento',  
204 |         'dados_incompletos',  
205 |         'aguardando_pagamento',  
206 |         'pagamento_confirmado',  
207 |         'origem_interna_em_validacao',  
208 |         'compra_interna_pendente',  
209 |         'compra_interna_realizada',  
210 |         'aguardando_postagem_envio',  
211 |         'rastreo_recebido',  
212 |         'em_transito',
```

create_checkout_order - RPC transacional do checkout

```
450 | create or replace function public.create_checkout_order(  
451 |     p_user_id uuid,  
452 |     p_customer_snapshot jsonb,  
453 |     p_address_snapshot jsonb,  
454 |     p_items jsonb,  
455 |     p_totals jsonb,  
456 |     p_payment jsonb,  
457 |     p_shipping jsonb,  
458 |     p_message text,  
459 |     p_consent_snapshot jsonb,  
460 |     p_request_context jsonb default '{}'::jsonb  
461 | )  
462 | returns jsonb  
463 | language plpgsql  
464 | as $$  
465 | declare  
466 |     v_auth_user_id uuid := auth.uid();  
467 |     v_consent_id uuid := null;  
468 |     v_item jsonb;  
469 |     v_order_id uuid;  
470 |     v_order_number text;  
471 | begin  
472 |     if p_user_id is null and v_auth_user_id is not null then  
473 |         p_user_id := v_auth_user_id;  
474 |     end if;  
475 |  
476 |     if p_user_id is not null and v_auth_user_id is not null and p_user_id <>  
477 |         v_auth_user_id then  
478 |         raise exception 'Nao e permitido criar pedido para outro usuario.';  
479 |     end if;  
480 |  
481 |     if jsonb_typeof(p_items) <> 'array' or jsonb_array_length(p_items) = 0 then  
482 |         raise exception 'Pedido sem itens.';  
483 |     end if;  
484 |  
485 |     if p_user_id is not null and coalesce((p_consent_snapshot ->> 'accepted')::boolean,  
486 |         false) then  
487 |         insert into public.assisted_purchase_consents (  
488 |             user_id,  
489 |             consent_version,  
490 |             consent_text,  
491 |             purchase_data_use_authorized,  
492 |             ip_address,  
493 |             user_agent  
494 |         )  
495 |         values (  
496 |             p_user_id,
```

A API chama esta funcao para gravar pedido, itens, pagamento e auditoria dentro do banco.

RLS

As migrations habilitam Row Level Security nas tabelas publicas. Cliente autenticado so ve/insere seus proprios perfis, enderecos, pedidos e avaliacoes. Dados internos de fornecedor e custos ficam para service_role/admin.

Storage buckets de imagens

```
72 | insert into storage.buckets (
73 |   id,
74 |   name,
75 |   public,
76 |   file_size_limit,
77 |   allowed_mime_types
78 | ) values (
79 |   'product-images',
80 |   'product-images',
81 |   true,
82 |   5242880,
83 |   array['image/jpeg', 'image/png', 'image/webp', 'image/gif']::text[]
84 | )
85 | on conflict (id) do update set
86 |   public = excluded.public,
87 |   file_size_limit = excluded.file_size_limit,
88 |   allowed_mime_types = excluded.allowed_mime_types;
89 |
90 | alter table public.orders
91 |   add column if not exists discount_snapshot jsonb not null default '{} '::jsonb;
92 |
93 | alter table public.order_items
94 |   add column if not exists unit_cost_cents integer check (unit_cost_cents is null or
95 |     unit_cost_cents >= 0),
96 |   add column if not exists subtotal_cost_cents integer check (subtotal_cost_cents is
97 |     null or subtotal_cost_cents >= 0);
98 |
99 | create or replace function public.redeem_catalog_coupon(p_code text)
100 | returns void
101 | language plpgsql
```

O projeto usa buckets publicos/controlados para imagens de produtos e fotos de avaliacoes.

5. Catálogo e vitrine

O catálogo nasce no Supabase, mas a aplicação converte linhas SQL em objetos de domínio JavaScript. Antes de chegar ao cliente, metadados internos como custo, margem e origem de compra são removidos.

src/catalog/supabase-catalog-core.js - leitura do catálogo publicado

```
1 | export function toCatalogProduct(row) {
2 |   const variationImages = Array.isArray(row.variation_images)
3 |     ? row.variation_images
4 |       .map((group) => ({
5 |         imageUrls: Array.isArray(group?.image_urls)
6 |           ? group.image_urls.filter((url) => typeof url === "string" && url.trim())
7 |           : [],
8 |         variation: String(group?.variation ?? "").trim()
9 |       }))
10 |     : [];
11 |   return {
12 |     id: row.id,
13 |     slug: row.slug,
14 |     name: row.name,
15 |     storefrontCategoryIds: row.storefront_category_ids ?? [],
16 |     productFamily: row.product_family,
17 |     bikeModelScope: row.bike_model_scope ?? [],
18 |     priceCents: row.price_cents,
19 |     currency: row.currency,
20 |     variations: row.variations ?? [],
21 |     availability: row.availability,
22 |     leadTimeDays: row.lead_time_days,
23 |     shippingClass: row.shipping_class,
24 |     imageUrls: row.image_urls ?? [],
25 |     variationImages,
26 |     checkoutChannel: row.checkout_channel,
27 |     internalPurchaseSource: row.internal_purchase_source ?? {},
28 |     notes: row.notes ?? "",
29 |     variationStock: (row.catalog_variation_stock ?? []).map((stock) => ({
30 |       quantity: stock.quantity,
31 |       variation: stock.variation
32 |     }))
33 |   };
34 | }
```

src/catalog/index.js - remoção de metadados internos

```
16 |   storefrontCategories,
17 |   technicalFamilies
18 | };
19 |
20 | export function getStorefrontMenu(products = catalogProducts) {
21 |   return storefrontCategories.map((category) => ({
22 |     ...category,
23 |     productCount: products.filter((product) =>
24 |       product.storefrontCategoryIds.includes(category.id))
25 |       .length
26 |     ));
27 | }
28 |
29 | export function toPublicCatalogProduct(product) {
30 |   const {
31 |     costCents: _costCents,
32 |     internalPurchaseCandidates: _internalPurchaseCandidates,
33 |     internalPurchaseSource: _internalPurchaseSource,
```

src/admin/catalog-admin.js - produto criado pelo admin

```
218 | function collectProductPayload(formData) {
219 |   const name = cleanString(formData.get("name"), 180);
220 |   const previousId = cleanString(formData.get("productId"), 160);
221 |   const persistenceMode = previousId ? "update" : "create";
222 |   const slug = slugify(formData.get("slug") || name);
223 |   const id = previousId || slug;
224 |   const storefrontCategoryIds = getSelectedCategoryIds(formData);
225 |   const productFamily = cleanString(formData.get("productFamily"), 80);
226 |   const priceCents = parseAdminMoneyToCents(formData.get("price"));
227 |   const costCents = parseAdminMoneyToCents(formData.get("cost"), { allowZero: true });
228 |   const {
229 |     stock: variationStock,
230 |     variationImageTokens,
231 |     variations
232 |   } = collectAdminVariationInventory(formData);
233 |   const usesVariationCards = formData.get("variationCards") !== null;
234 |   const imageUrls = splitImageUrls(formData.get("imageUrls"));
235 |   const imageOrderTokens = splitImageOrderTokens(formData.get("imageOrder"));
236 |   const bikeModelScope = splitList(formData.get("bikeModelScope"), {
237 |     maxItems: 8,
238 |     maxLength: 80
239 |   });
240 |
241 |   if (!name) {
242 |     throw new Error("Informe o nome do produto.");
243 |   }
244 |
245 |   if (!id || !slug) {
246 |     throw new Error("Informe um slug/ID valido para o produto.");
247 |   }
248 |
249 |   if (storefrontCategoryIds.length === 0) {
250 |     throw new Error("Selecione pelo menos uma categoria.");
251 |   }
252 |
253 |   if (!technicalFamilies.includes(productFamily)) {
254 |     throw new Error("Familia tecnica invalida.");
255 |   }
256 |
257 |   if (!Number.isInteger(priceCents)) {
258 |     throw new Error("Informe um preco do cliente valido.");
259 |   }
260 |
261 |   if (cleanString(formData.get("cost"), 40) && !Number.isInteger(costCents)) {
262 |     throw new Error("Informe um preco real valido ou deixe vazio.");
263 |   }
264 |
265 |   if (variations.length === 0) {
266 |     throw new Error("Informe pelo menos uma variacao.");
267 |   }
```

O admin cria/edita produtos, custos, variacoes, categorias e URLs de imagens; depois revalida as rotas estaticas.

6. Checkout, WhatsApp e pedidos

O checkout e desenhado para nao confiar no navegador. O cliente envia o carrinho, mas o backend recarrega o catalogo, valida produtos/variacoes, recalcula precos e so entao monta o pedido e a mensagem.

app/api/checkout/whatsapp/route.js - ponto de entrada do checkout

```
71 | export async function POST(request) {
72 |   if (!isSameOriginRequest(request)) {
73 |     return errorResponse("Origem da requisicao nao permitida.", 403);
74 |   }
75 |
76 |   if (!isJsonRequest(request)) {
77 |     return errorResponse("Envie o checkout como application/json.", 415);
78 |   }
79 |
80 |   const phoneNumber = getConfiguredWhatsAppNumber();
81 |
82 |   if (!phoneNumber) {
83 |     return errorResponse("WhatsApp Business indisponivel no momento.", 503);
84 |   }
85 |
86 |   const serviceSupabase = createServiceRoleSupabaseClient();
87 |   const rateLimit = await consumeRateLimit({
88 |     ...rateLimitProfiles.checkout,
89 |     identifier: getRequestIp(request),
90 |     supabase: serviceSupabase
91 |   });
92 |
93 |   if (!rateLimit.allowed) {
94 |     logServerEvent("warn", "checkout_rate_limit_blocked", {
95 |       retryAfterSeconds: rateLimit.retryAfterSeconds,
96 |       unavailable: rateLimit.unavailable
97 |     });
98 |
99 |     return createRateLimitResponse(rateLimit, {
100 |       rateLimitedMessage: "Muitas tentativas de checkout. Tente novamente em
instantes."
101 |     });
102 |   }
103 |
104 |   let payload;
105 |
106 |   try {
107 |     payload = await request.json();
108 |   } catch {
109 |     logServerEvent("warn", "checkout_validation_error", {
110 |       reason: "invalid_json"
111 |     });
112 |     return errorResponse("Envie um JSON valido para finalizar o pedido.", 400);
113 |   }
114 |
115 |   const storeName = process.env.NEXT_PUBLIC_STORE_NAME ?? "TSZR15";
116 |   const userSupabase = await createServerSupabaseClient();
117 |   const supabase = serviceSupabase ?? userSupabase;
118 |
119 |   let draft;
120 |
121 |   try {
122 |     const catalog = await getSupabaseCatalogProducts();
123 |     const products = await attachInternalProductCosts(catalog.products,
serviceSupabase);
124 |     const baseDraft = buildCheckoutOrderDraft(payload, { products, storeName });
125 |     const coupon = await resolveCheckoutCoupon({
126 |       cartItems: baseDraft.cartItems,
127 |       code: payload?.couponCode,
128 |       supabase: serviceSupabase
129 |     });
130 |
131 |     draft = coupon ? buildCheckoutOrderDraft(payload, { coupon, products, storeName })
: baseDraft;
132 |
133 |     const unavailableItems = draft.cartItems.filter((item) => {
134 |       const stock = getVariationStockStatus(
135 |         products.find((product) => product.id === item.id),
136 |         item.variation
137 |       );
```

A rota aplica rate limit, parseia JSON, pega telefone da loja, cria cliente Supabase e monta o draft do pedido.

src/checkout/order-backend.js - persistencia via RPC

```
299 | export async function persistCheckoutOrder({ draft, requestContext = {}, supabase, user
    | }) {
300 |   if (!supabase) {
301 |     return {
302 |       reason: "Supabase nao configurado.",
303 |       saved: false
304 |     };
305 |   }
306 |
307 |   // create_checkout_order e executavel apenas pelo service role; sem a key o
308 |   // caminho autenticado falharia com permission denied no FOR UPDATE.
309 |   if (!getSupabaseServiceRoleKey()) {
310 |     return {
311 |       reason: "Pedido precisa de SUPABASE_SERVICE_ROLE_KEY no servidor.",
312 |       saved: false
313 |     };
314 |   }
315 |
316 |   const { data, error } = await supabase.rpc("create_checkout_order", {
317 |     p_address_snapshot: draft.addressSnapshot,
318 |     p_consent_snapshot: draft.consentSnapshot,
319 |     p_customer_snapshot: draft.customerSnapshot,
320 |     p_items: draft.databaseItems,
321 |     p_message: draft.message,
322 |     p_payment: draft.payment,
323 |     p_request_context: requestContext,
324 |     p_shipping: draft.shipping,
325 |     p_totals: draft.totals,
326 |     p_user_id: user?.id ?? null
327 |   });
328 |
329 |   if (error) {
330 |     const stockError = toCheckoutStockError(error.message, draft.databaseItems);
331 |
332 |     if (stockError) {
333 |       throw stockError;
334 |     }
335 |
336 |     throw new CheckoutPersistenceError(error.message);
```

src/checkout/whatsapp.js - mensagem e URL wa.me

```
131 | export function buildWhatsAppCheckoutUrl({ phoneNumber, message }) {
132 |   const normalizedPhoneNumber = normalizePhoneNumber(phoneNumber);
133 |
134 |   if (!normalizedPhoneNumber) {
135 |     return "";
136 |   }
137 |
138 |   return `https://wa.me/${normalizedPhoneNumber}?text=${encodeURIComponent(message)}`;
139 | }
```

encodeURIComponent evita quebrar a URL com quebras de linha, acentos e simbolos monetarios.

src/components/catalog/cart-checkout.js - envio pelo browser

```
// marcador nao encontrado em src/components/catalog/cart-checkout.js: async function
submitCheckout
```

Depois que a API responde, o browser limpa o carrinho e abre o WhatsApp em nova aba.

7. Cupons, rate limit e logs

Cupons e checkout usam protecoes de abuso. O identificador de rate limit e hashado antes de ir ao banco, e em producao o sistema falha fechado se o Supabase de rate limit estiver indisponivel.

src/checkout/coupons.js - normalizacao e regras do cupom

```
1 | import { CheckoutValidationError } from "../order-backend.js";
2 |
3 | export function normalizeCouponCode(value) {
4 |   return String(value ?? "").
5 |     .trim()
6 |     .toUpperCase()
7 |     .replace(/[^A-Z0-9_-]/g, "")
8 |     .slice(0, 40);
9 | }
10 |
11 | export function getCouponRateLimitIdentifiers({ couponCode, ip }) {
12 |   const normalizedIp = String(ip ?? "unknown").trim() || "unknown";
13 |   const normalizedCode = normalizeCouponCode(couponCode) || "empty";
14 |
15 |   return {
16 |     code: `${normalizedIp}:${normalizedCode}`,
17 |     global: normalizedIp
18 |   };
19 | }
20 |
21 | export function toPublicCheckoutCoupon(coupon) {
22 |   if (!coupon) {
23 |     return null;
24 |   }
25 |
26 |   return {
27 |     code: coupon.code,
28 |     discountCents: coupon.discountCents,
29 |     discountPercent: coupon.discountPercent,
30 |     discountType: coupon.discountType
31 |   };
32 | }
33 |
34 | function cents(value) {
35 |   return Number.isInteger(value) && value > 0 ? value : 0;
36 | }
37 |
38 | function hasIntersection(left = [], right = []) {
39 |   const rightSet = new Set(right);
40 |   return left.some((value) => rightSet.has(value));
41 | }
42 |
43 | function getDiscountableSubtotal(cartItems, coupon) {
44 |   const productIds = coupon.appliesToProductIds ?? [];
45 |   const categoryIds = coupon.appliesToCategoryIds ?? [];
46 |   const isGlobalCoupon = productIds.length === 0 && categoryIds.length === 0;
47 |
48 |   return cartItems.reduce((total, item) => {
49 |     const itemSubtotal = item.priceCents * item.quantity;
50 |
51 |     if (isGlobalCoupon) {
52 |       return total + itemSubtotal;
53 |     }
54 |
55 |     if (productIds.includes(item.id)) {
56 |       return total + itemSubtotal;
57 |     }
58 |
59 |     if (hasIntersection(item.storefrontCategoryIds, categoryIds)) {
60 |       return total + itemSubtotal;
61 |     }
62 |
63 |     return total;
64 |   }, 0);
65 | }
66 |
67 | export function toCheckoutCoupon(row, cartItems, now = new Date()) {
```

src/lib/rate-limit.js - perfis de limitacao

```
3 | const localRateLimitStore = new Map();
4 | const productionEnvironments = new Set(["production", "preview"]);
5 |
6 | export const rateLimitProfiles = {
7 |   adminLogin: {
8 |     blockSeconds: 15 * 60,
9 |     limit: 5,
10 |    scope: "admin-login",
11 |    windowSeconds: 15 * 60
12 |  },
13 |  checkout: {
14 |    blockSeconds: 60,
15 |    limit: 10,
16 |    scope: "checkout-whatsapp",
17 |    windowSeconds: 60
18 |  },
19 |  coupon: {
20 |    blockSeconds: 60,
21 |    limit: 20,
22 |    scope: "coupon-validate-code",
23 |    windowSeconds: 60
24 |  },
25 |  couponGlobal: {
26 |    blockSeconds: 60,
27 |    limit: 30,
28 |    scope: "coupon-validate-ip",
29 |    windowSeconds: 60
30 |  },
31 |  tracking: {
```

supabase/migrations/...api_rate_limits.sql - tabela privada e RPC

```
1 | create schema if not exists private;
2 |
3 | create table if not exists private.api_rate_limits (
4 |   scope text not null,
5 |   identifier_hash text not null,
6 |   window_started_at timestamptz not null default now(),
7 |   attempt_count integer not null default 0 check (attempt_count >= 0),
8 |   blocked_until timestamptz,
9 |   updated_at timestamptz not null default now(),
10 |   primary key (scope, identifier_hash)
11 | );
12 |
13 | alter table private.api_rate_limits enable row level security;
14 |
15 | revoke all on schema private from public, anon, authenticated;
16 | grant usage on schema private to service_role;
17 |
18 | revoke all on table private.api_rate_limits from public, anon, authenticated;
19 | grant select, insert, update, delete on table private.api_rate_limits to service_role;
20 |
21 | create index if not exists api_rate_limits_updated_at_idx
22 | on private.api_rate_limits (updated_at);
23 |
24 | create or replace function public.consume_rate_limit(
25 |   p_scope text,
26 |   p_identifier_hash text,
27 |   p_limit integer,
28 |   p_window_seconds integer,
29 |   p_block_seconds integer default null,
30 |   p_increment boolean default true
31 | )
32 | returns jsonb
33 | language plpgsql
34 | set search_path = ''
35 | as $$
36 | declare
37 |   v_now timestamptz := now();
38 |   v_row private.api_rate_limits%rowtype;
```

src/lib/logger.js - redacao de dados sensiveis

```
1 | const censor = "[redacted]";
2 | const sensitiveKeyPattern =
3 |
4 | /authorization|cookie|token|password|email|phone|whatsapp|tax|cpf|cnpj|customer|payload|message|url/i;
5 |
6 | const logLevels = Object.freeze({
7 |   trace: 10,
8 |   debug: 20,
9 |   info: 30,
10 |   warn: 40,
11 |   error: 50,
12 |   fatal: 60,
13 | });
14 |
15 | const consoleMethods = Object.freeze({
16 |   trace: "debug",
17 |   debug: "debug",
18 |   info: "info",
19 |   warn: "warn",
20 |   error: "error",
21 |   fatal: "error",
22 | });
23 |
24 | export function redactSensitive(value) {
25 |   if (Array.isArray(value)) {
26 |     return value.map((item) => redactSensitive(item));
27 |   }
28 |
29 |   if (!value || typeof value !== "object") {
30 |     return value;
31 |   }
32 |
33 |   return Object.fromEntries(
34 |     Object.entries(value).map(([key, entryValue]) => [
35 |       key,
36 |       sensitiveKeyPattern.test(key) ? censor : redactSensitive(entryValue),
37 |     ]),
38 |   );
39 | }
40 |
41 | function shouldLog(level) {
42 |   const configuredLevel = Object.hasOwn(logLevels, process.env.LOG_LEVEL)
43 |     ? process.env.LOG_LEVEL
44 |     : "info";
45 |
46 |   return logLevels[level] >= logLevels[configuredLevel];
47 | }
48 |
49 | function normalizeLogArguments(input, message) {
50 |   if (typeof input === "string" && message === undefined) {
51 |     return { details: {}, message: input };
52 |   }
53 |
54 |   if (input instanceof Error) {
55 |     return {
56 |       details: {
57 |         err: {
58 |           message: input.message,
```

8. Autenticacao, conta e admin

A autenticacao de cliente usa Supabase Auth. A area admin nao usa conta Supabase normal: ela usa um token TSZR15_ADMIN_TOKEN, valida por hash em tempo constante e cria um cookie assinado com HMAC.

app/auth/actions.js - Server Actions de login/cadastro

```
354 | export async function signInAction(formData) {
355 |   const email = formValue(formData, "email");
356 |   const password = formValue(formData, "password");
357 |   const nextPath = getSafeAuthRedirectPath(formValue(formData, "next"), "/conta");
358 |
359 |   if (email.toLowerCase() === "admin") {
360 |     const ratelimitContext = await checkAdminLoginRateLimit();
361 |
362 |     if (!ratelimitContext.result.allowed) {
363 |       const message = ratelimitContext.result.unavailable
364 |         ? "Protecao administrativa indisponivel. Tente novamente em instantes."
365 |         : "Muitas tentativas administrativas. Aguarde alguns minutos antes de tentar de
novo.";
366 |
367 |       redirectWithError("/entrar", message, "/admin");
368 |     }
369 |
370 |     if (!isAdminTokenConfigured()) {
371 |       logServerEvent("error", "admin_login_token_missing");
372 |       redirectWithError(
373 |         "/entrar",
374 |         "Token administrativo ausente no servidor. Configure TSZR15_ADMIN_TOKEN no
ambiente de producao.",
375 |         "/admin"
376 |       );
377 |     }
378 |
379 |     const isValidAdmin = await startAdminSession(password);
380 |
381 |     if (!isValidAdmin) {
382 |       const failedAttempt = await recordFailedAdminLogin(ratelimitContext);
383 |
384 |       if (!failedAttempt.allowed) {
385 |         redirectWithError(
386 |           "/entrar",
387 |           "Muitas tentativas administrativas. Aguarde alguns minutos antes de tentar de
novo.",
388 |           "/admin"
389 |         );
390 |       }
391 |
392 |       redirectWithError("/entrar", "Senha administrativa invalida.", "/admin");
393 |     }
394 |
395 |     await resetRateLimit({
396 |       ...ratelimitProfiles.adminLogin,
397 |       identifier: ratelimitContext.identifier,
398 |       supabase: ratelimitContext.supabase
399 |     });
400 |     logServerEvent("info", "admin_login_success");
401 |     redirect("/admin");
402 |   }
403 |
404 |   const supabase = await createServerSupabaseClient();
405 |
406 |   if (!supabase) {
407 |     const status = getSupabaseConfigStatus();
408 |     const missing = status.missing.length ? `Faltando: ${status.missing.join("; ")}`
: "";
409 |     redirectWithError(
410 |       "/entrar",
411 |       `Configure as variaveis do Supabase antes de entrar.${missing}`,
412 |       nextPath
413 |     );
414 |   }
415 | }
```

src/admin/admin-session.js - cookie admin assinado

```
1 | import { createHash, createHmac, timingSafeEqual } from "crypto";
2 |
3 | import {
4 |   ADMIN_SESSION_COOKIE,
5 |   ADMIN_SESSION_MAX_AGE_SECONDS,
6 |   ADMIN_SESSION_VERSION,
7 |   ADMIN_TOKEN_MIN_LENGTH
8 | } from "../admin-session-constants.js";
9 |
10 | export { ADMIN_SESSION_COOKIE, ADMIN_SESSION_MAX_AGE_SECONDS };
11 |
12 | export function getConfiguredAdminToken() {
13 |   return process.env.TSZR15_ADMIN_TOKEN ?? "";
14 | }
15 |
16 | export function isAdminTokenValueConfigured(token = getConfiguredAdminToken()) {
17 |   return token.length >= ADMIN_TOKEN_MIN_LENGTH;
18 | }
19 |
20 | function hashValue(value) {
21 |   return createHash("sha256").update(String(value ?? "")).digest("hex");
22 | }
23 |
24 | function signAdminSessionPayload(payload, token) {
25 |   return createHmac("sha256", token).update(payload).digest("hex");
26 | }
27 |
28 | function parseAdminSessionValue(sessionValue) {
29 |   const [version, expiresAtValue, signature, ...extra] = String(sessionValue ??
30 |     "").split(".");
31 |   const expiresAt = Number(expiresAtValue);
32 |
33 |   if (
34 |     extra.length > 0 ||
35 |     version !== ADMIN_SESSION_VERSION ||
36 |     !Number.isSafeInteger(expiresAt) ||
37 |     !/^[a-f0-9]{64}$/i.test(signature ?? "")
38 |   ) {
39 |     return null;
40 |   }
41 |
42 |   return {
43 |     expiresAt,
44 |     signature,
45 |     version
46 |   };
47 | }
48 |
49 | function safeEqual(left, right) {
50 |   if (!left || !right || left.length !== right.length) {
51 |     return false;
52 |   }
53 |   return timingSafeEqual(Buffer.from(left), Buffer.from(right));
54 | }
```

app/admin/actions.js - Server Action administrativa

```
1 | "use server";
2 |
3 | import { headers } from "next/headers";
4 | import { redirect } from "next/navigation";
5 | import { revalidatePath } from "next/cache";
6 |
7 | import { getAdminActionErrorState } from "@src/admin/admin-action-error.js";
8 | import { clearAdminSession, isAdminSessionValid } from "@src/admin/admin-auth.js";
9 | import {
10 |   archiveAdminCoupon,
11 |   archiveAdminCatalogProduct,
12 |   upsertAdminCoupon,
13 |   upsertAdminCatalogProduct
14 | } from "@src/admin/catalog-admin.js";
15 | import {
16 |   createAdminManualOrder,
17 |   setAdminInternalOrderStatus,
18 |   updateAdminOrderOperation
19 | } from "@src/admin/order-admin.js";
20 | import { moderateOrderReview } from "@src/reviews/order-reviews.js";
21 | import { revalidateCatalogPaths } from "@src/catalog/revalidation.js";
22 | import { isSameOriginRequest } from "@src/security/origin.js";
23 |
24 | function formValue(formData, key) {
25 |   return String(formData.get(key) ?? "").trim();
26 | }
27 |
28 | function redirectWithError(path, message) {
29 |   const separator = path.includes("?") ? "&" : "?";
30 |   redirect(`${path}${separator}error=${encodeURIComponent(message)}`);
31 | }
32 |
33 | function revalidateAdminPanel() {
34 |   revalidatePath("/admin", "layout");
35 | }
36 |
37 | // Toda saida de erro do painel passa por aqui: mensagem de validacao segue
38 | // intacta, detalhe de driver vira mensagem generica + referencia de log.
39 | function getActionErrorMessage(error) {
40 |   return getAdminActionErrorState(error).message;
41 | }
42 |
43 | async function isSameOriginAdminRequest() {
44 |   const headerStore = await headers();
45 |
46 |   return isSameOriginRequest(headerStore);
47 | }
48 |
49 | export async function adminSignOutAction() {
50 |   if (!(await isAdminSessionValid()) || !(await isSameOriginAdminRequest())) {
51 |     redirect("/entrar?next=/admin");
52 |   }
53 |
54 |   await clearAdminSession();
```

Por que Server Actions?

Formulários de conta e admin podem chamar funções do servidor diretamente. Isso permite validar sessão, checar origem, atualizar Supabase e chamar `revalidatePath` sem expor chaves ao navegador.

9. Rastreo e avaliacoes

Rastreo publico e avaliacoes usam dados internos, mas filtram a exposicao. O cliente consulta por numero do pedido e contato; fotos de avaliacao ficam no Storage e sao servidas por URLs assinadas quando necessario.

src/tracking/order-tracking.js - consulta segura do rastreio

```
73 | export async function findPublicOrderTracking({ contact, orderNumber, supabase }) {
74 |   const client = supabase ?? createServiceRoleSupabaseClient();
75 |
76 |   if (!client) {
77 |     return {
78 |       reason: "tracking_not_configured",
79 |       status: "setup-required"
80 |     };
81 |   }
82 |
83 |   const cleanOrderNumber = cleanString(orderNumber, 80).toUpperCase();
84 |
85 |   if (!cleanOrderNumber || !cleanString(contact, 120)) {
86 |     return {
87 |       reason: "missing-fields",
88 |       status: "empty"
89 |     };
90 |   }
91 |
92 |   const { data: order, error } = await client
93 |     .from("orders")
94 |     .select(".*")
95 |     .eq("order_number", cleanOrderNumber)
96 |     .limit(1)
97 |     .maybeSingle();
98 |
99 |   if (error) {
100 |     throw new Error(error.message);
101 |   }
102 |
103 |   if (!order || !contactMatchesOrder(order, contact)) {
104 |     return {
105 |       reason: "not-found",
106 |       status: "not-found"
107 |     };
108 |   }
109 |
110 |   const [
111 |     { data: items, error: itemsError },
112 |     { data: supplierPurchases, error: supplierError },
113 |     { data: trackingEvents, error: trackingError }
114 |   ] = await Promise.all([
115 |     client
116 |       .from("order_items")
117 |       .select("product_name, variation, quantity")
118 |       .eq("order_id", order.id)
119 |       .order("created_at"),
120 |     client
121 |       .from("supplier_purchases")
122 |       .select("carrier, source_eta, tracking_code")
123 |       .eq("order_id", order.id)
124 |       .order("created_at")
125 |       .limit(1),
126 |     client
127 |       .from("supplier_tracking_events")
128 |       .select("id, event_status, event_at, location, description, created_at")
129 |       .eq("order_id", order.id)
130 |       .order("event_at", { ascending: false })
131 |       .order("created_at", { ascending: false })
132 |   ]);
133 |
134 |   const firstError = itemsError ?? supplierError ?? trackingError;
```

src/reviews/order-reviews.js - upload e assinatura de fotos

```
// marcador nao encontrado em src/reviews/order-reviews.js: async function uploadReviewPhotos
```

app/rastreio/page.js - pagina publica de consulta

```
13 | export default async function TrackingPage() {  
14 |   const supabase = await createServerSupabaseClient();  
15 |   const snapshot = await getCurrentCustomerSnapshot(supabase);  
16 |  
17 |   return (  
18 |     <main className={cx(globalStyles, "page-shell auth-page tracking-page")}>  
19 |       <SiteHeader user={snapshot.user} />  
20 |       <TrackingLookup />  
21 |     </main>  
22 |   );  
23 | }
```


10. Estilo, assets e APIs do navegador

A camada visual usa CSS global em `app/globals.css`, CSS Module pontual para componente de senha, imagens em `public/brand` e URLs de Storage Supabase para assets remotos.

Recurso	Arquivo	Uso
CSS global	<code>app/globals.css</code>	Layout da loja, botoes, cards, admin, rastreo, carrinho e responsividade.
CSS Module	<code>src/components/form/password-input.module.css</code>	Escopo local do input de senha.
public assets	<code>public/brand/*.png</code>	Logo e imagens de marca versionadas no repo.
Supabase Storage	<code>heroBoardSrc</code> e <code>buckets/product-images/review-photos</code>	Imagens que podem vir do painel/admin ou do bucket publico.
Web APIs	<code>localStorage</code> , <code>fetch</code> , <code>AbortController</code> , <code>window.open</code>	Carrinho, chamadas internas, consulta de CEP e abertura do WhatsApp.
Intl	<code>Intl.NumberFormat</code> , <code>Intl.DateTimeFormat</code>	Moeda e datas em pt-BR.

src/customer/cep-lookup.js - ViaCEP e AbortController

```
1 | import { digitsOnly, sanitizeCep, sanitizeState } from "../field-validation.js";
2 |
3 | const viaCepBaseUrl = "https://viacep.com.br/ws";
4 |
5 | function cleanText(value) {
6 |   return String(value ?? "").trim();
7 | }
8 |
9 | export function getCepDigits(value) {
10 |   return digitsOnly(value).slice(0, 8);
11 | }
12 |
13 | export function canLookupCep(value) {
14 |   return getCepDigits(value).length === 8;
15 | }
16 |
17 | export function buildViaCepLookupUrl(value) {
18 |   const cepDigits = getCepDigits(value);
19 |
20 |   if (cepDigits.length !== 8) {
21 |     return "";
22 |   }
23 |
24 |   return `${viaCepBaseUrl}/${cepDigits}.json`;
25 | }
26 |
27 | export function normalizeViaCepAddress(data) {
28 |   if (!data || data.erro) {
29 |     return null;
30 |   }
31 |
32 |   const state = sanitizeState(data.uf);
33 |
34 |   return {
35 |     cep: sanitizeCep(data.cep),
36 |     city: cleanText(data.localidade),
37 |     district: cleanText(data.bairro),
38 |     state,
39 |     street: cleanText(data.logradouro)
40 |   };
41 | }
42 |
43 | export function formatCepAddressLine(address) {
44 |   if (!address) {
45 |     return "";
46 |   }
47 |
48 |   const cityLine = [address.city, address.state].filter(Boolean).join("/");
49 |
50 |   return [address.street, address.district, cityLine].filter(Boolean).join(" - ");
51 | }
52 |
53 | export async function fetchCepAddress(value, { signal } = {}) {
54 |   const lookupUrl = buildViaCepLookupUrl(value);
55 |
56 |   if (!lookupUrl) {
57 |     return null;
58 |   }
59 |
60 |   const response = await fetch(lookupUrl, { signal });
61 |
62 |   if (!response.ok) {
63 |     throw new Error("Nao foi possivel consultar o CEP agora.");
64 |   }
65 |
66 |   return normalizeViaCepAddress(await response.json());
```

O ViaCEP nao exige chave. O projeto monta a URL apenas quando o CEP tem 8 digitos, normaliza a resposta e deixa o componente cliente abortar requests antigos quando o usuario altera o campo rapidamente.

11. Scripts, testes e qualidade

O projeto usa scripts npm simples para desenvolvimento, build, lint, formatacao, testes e sincronizacao de catalogo. Os testes usam node:test e assert/strict, sem dependencia extra de test runner.

package.json - scripts principais

```
6 | "scripts": {
7 |   "dev": "next dev",
8 |   "build": "next build",
9 |   "build:cf": "opennextjs-cloudflare build",
10 |   "bundle:sites": "wrangler deploy --dry-run --outfile
    .sites-worker-bundle/upload.multipart --config .openai/sites-bundle-wrangler.json",
11 |   "build:sites": "npm run build:cf && npm run bundle:sites && node
    scripts/prepare-sites-build.mjs",
12 |   "deploy": "opennextjs-cloudflare build && opennextjs-cloudflare deploy",
13 |   "preview": "opennextjs-cloudflare build && opennextjs-cloudflare preview",
14 |   "format": "prettier --write \"app/api/**/*.js\" \"app/auth/actions.js\"
    \"app/page.js\" \"app/catalogo/page.js\" \"app/pedido/page.js\"
    \"app/produto/[slug]/page.js\" \"src/components/catalog/*.js\" \"src/lib/*.js\"
    \"tests/rate-limit.test.mjs\" \"*.json,mjs\"\"",
15 |   "format:check": "prettier --check \"app/api/**/*.js\" \"app/auth/actions.js\"
    \"app/page.js\" \"app/catalogo/page.js\" \"app/pedido/page.js\"
    \"app/produto/[slug]/page.js\" \"src/components/catalog/*.js\" \"src/lib/*.js\"
    \"tests/rate-limit.test.mjs\" \"*.json,mjs\"\"",
16 |   "lint": "eslint app src tests scripts next.config.mjs",
17 |   "start": "next start",
18 |   "clone:preview": "node scripts/clone-supabase-preview.mjs",
19 |   "sync:catalog": "node scripts/sync-supabase-catalog.mjs",
20 |   "sync:catalog:preview": "node scripts/sync-supabase-catalog.mjs --target preview",
21 |   "optimize:brand-assets": "node scripts/optimize-brand-assets.mjs",
```

scripts/validate-catalog.mjs - validacao local do catalogo

```
1 | import { catalogProducts, rejectedCatalogProducts, validateCatalog } from
    "../src/catalog/index.js";
2 |
3 | const issues = validateCatalog();
4 |
5 | if (issues.length > 0) {
6 |   console.error("Catalog validation failed:");
7 |   for (const issue of issues) {
8 |     console.error(`- ${issue}`);
9 |   }
10 |   process.exit(1);
11 | }
12 |
13 | console.log(`Catalog validation passed for ${catalogProducts.length} published
    products.`);
14 | console.log(`Rejected products: ${rejectedCatalogProducts.length}.`);
```

scripts/sync-supabase-catalog.mjs - seed/upsert do Supabase

```
171 | async function main() {
172 |   loadLocalEnv();
173 |
174 |   if (targetArg) {
175 |     process.env.SUPABASE_RUNTIME_TARGET = targetArg;
176 |   }
177 |
178 |   const catalogIssues = validateCatalog();
179 |
180 |   if (catalogIssues.length > 0) {
181 |     throw new Error(`Catalogo local invalido:\n${catalogIssues.join("\n")}`);
182 |   }
183 |
184 |   const categoryRows = buildCatalogCategoryRows();
185 |   const productRows = buildCatalogProductRows();
186 |   const relationRows = buildCatalogProductCategoryRows();
187 |
188 |   if (dryRun) {
189 |     console.log(
190 |       `Dry run: ${categoryRows.length} categorias, ${productRows.length} produtos e
191 |       ${relationRows.length} vinculos prontos para sincronizar.`
192 |     );
193 |     return;
194 |   }
195 |   if (sqlOutputPath) {
196 |     const outputPath = resolve(process.cwd(), sqlOutputPath);
197 |
198 |     mkdirSync(resolve(outputPath, ".."), { recursive: true });
199 |     writeFileSync(outputPath, buildSeedSql({ categoryRows, productRows, relationRows
200 |     )), "utf8");
201 |     console.log(`SQL de seed gerado em ${sqlOutputPath}.`);
202 |     return;
203 |   }
204 |   const url = getSupabaseUrl();
205 |   const serviceRoleKey = getSupabaseServiceRoleKey();
206 |
207 |   if (!url || !serviceRoleKey) {
208 |     throw new Error(
209 |       "Configure as envs Supabase do destino. Para preview use
210 |       NEXT_PUBLIC_SUPABASE_PREVIEW_URL e SUPABASE_PREVIEW_SERVICE_ROLE_KEY, ou rode com as envs
211 |       padrao."
212 |     );
213 |   }
214 |   if (getJwtRole(serviceRoleKey) === "anon") {
215 |     throw new Error(
216 |       "SUPABASE_SERVICE_ROLE_KEY esta com uma chave anon. Use a service_role real ou
217 |       gere SQL com: npm run sync:catalog -- --write-sql supabase/.temp/catalog-seed.sql"
218 |     );
219 |   }
220 |   const client = createClient(url, serviceRoleKey, {
221 |     auth: {
222 |       autoRefreshToken: false,
223 |       persistSession: false
224 |     }
225 |   });
```

tests/rate-limit.test.mjs - exemplo de teste node:test

```
1 | import assert from "node:assert/strict";
2 | import { afterEach, test } from "node:test";
3 |
4 | import { redactSensitive } from "../src/lib/logger.js";
5 | import {
6 |   clearLocalRateLimitStore,
7 |   consumeRateLimit,
8 |   getRequestIp,
9 |   hashRateLimitIdentifier,
10 |   rateLimitProfiles
11 | } from "../src/lib/rate-limit.js";
12 | import { createRateLimitResponse } from "../src/lib/rate-limit-response.js";
13 |
14 | afterEach(() => {
15 |   clearLocalRateLimitStore();
16 |   delete process.env.VERCEL_ENV;
17 | });
18 |
19 | test("rate limit permite requisicoes dentro da janela e bloqueia acima do limite",
20 |   async () => {
21 |     const profile = {
22 |       blockSeconds: 30,
23 |       limit: 2,
24 |       scope: "test-checkout",
25 |       windowSeconds: 60
26 |     };
27 |     const first = await consumeRateLimit({ ...profile, identifier: "127.0.0.1", supabase:
28 |       null });
29 |     const second = await consumeRateLimit({ ...profile, identifier: "127.0.0.1",
30 |       supabase: null });
31 |     const third = await consumeRateLimit({ ...profile, identifier: "127.0.0.1", supabase:
32 |       null });
33 |     assert.equal(first.allowed, true);
34 |     assert.equal(second.allowed, true);
35 |     assert.equal(third.allowed, false);
36 |     assert.equal(third.retryAfterSeconds, 30);
37 |   });
38 |
39 | test("IP do rate limit usa apenas headers confiaveis da Cloudflare", () => {
40 |   assert.equal(
41 |     getRequestIp(
42 |       new Headers({
43 |         "cf-connecting-ip": "203.0.113.10",
44 |         "true-client-ip": "203.0.113.11",
45 |         "x-forwarded-for": "198.51.100.200"
46 |       })
47 |     ),
48 |     "203.0.113.10"
49 |   );
50 | });
```

eslint.config.mjs - lint JavaScript/JSX

```
1 | import js from "@eslint/js";
2 | import prettier from "eslint-config-prettier/flat";
3 | import globals from "globals";
4 |
5 | export default [
6 |   {
7 |     ignores: [".next/**", "node_modules/**", "coverage/**", "supabase/.temp/**"]
8 |   },
9 |   js.configs.recommended,
10 |   {
11 |     languageOptions: {
12 |       ecmaVersion: "latest",
13 |       globals: {
14 |         ...globals.browser,
15 |         ...globals.node
16 |       },
17 |       parserOptions: {
18 |         ecmaFeatures: {
19 |           jsx: true
20 |         }
21 |       },
22 |       sourceType: "module"
23 |     },
24 |     rules: {
25 |       "no-control-regex": "off",
26 |       "no-useless-assignment": "warn",
27 |       "no-unused-vars": [
28 |         "warn",
29 |         {
30 |           argsIgnorePattern: "^_",
31 |           caughtErrorsIgnorePattern: "^_",
32 |           varsIgnorePattern: "^_"
33 |         }
34 |       ]
35 |     }
36 |   },
37 |   prettier
38 | ];
```

.github/workflows/secret-scan.yml - Gitleaks no CI

```
1 | name: Secret scan
2 |
3 | on:
4 |   pull_request:
5 |   push:
6 |     branches:
7 |       - main
8 |       - MODEL_WHATSBUIS
9 |   workflow_dispatch:
10 |
11 | permissions:
12 |   contents: read
13 |   pull-requests: read
14 |
15 | jobs:
16 |   gitleaks:
17 |     name: Gitleaks
18 |     runs-on: ubuntu-latest
19 |     steps:
20 |       - name: Checkout full history
21 |         uses: actions/checkout@v4
22 |         with:
23 |           fetch-depth: 0
24 |
25 |       - name: Install Gitleaks
26 |         shell: bash
27 |         run: |
28 |           set -e
29 |           set -x
30 |           set -uo pipefail
31 |           version="8.30.1"
32 |           asset="gitleaks_${version}_linux_x64.tar.gz"
33 |           base_url="https://github.com/gitleaks/gitleaks/releases/download/v${version}"
34 |           curl -sSfL "${base_url}/${asset}" -o "/tmp/${asset}"
35 |           curl -sSfL "${base_url}/gitleaks_${version}_checksums.txt" -o
36 |             /tmp/gitleaks_checksums.txt
```

12. Configuracao e deploy

A configuracao do projeto e feita por variaveis de ambiente. O arquivo .env.example mostra nomes esperados sem segredos reais. O deploy de producao roda em Cloudflare Workers via OpenNext (@opennextjs/cloudflare): npm run deploy faz o build (opennextjs-cloudflare build) e publica o worker; wrangler.jsonc define o worker, os assets estaticos e os bindings de KV (cache incremental), D1 (tag cache) e Durable Object (fila de revalidacao). O codigo ainda detecta Vercel Preview: prefere credenciais de Supabase Preview quando VERCEL_ENV=preview ou SUPABASE_RUNTIME_TARGET=preview.

.env.example - variaveis usadas pelo app

```
1 | NEXT_PUBLIC_WHATSAPP_BUSINESS_NUMBER=5511999999999
2 | WHATSAPP_BUSINESS_NUMBER=5511999999999
3 | NEXT_PUBLIC_STORE_NAME=TSZR15
4 | NEXT_PUBLIC_SITE_URL=https://www.tszr15-store.com.br
5 | NEXT_PUBLIC_SUPABASE_URL=
6 | NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
7 | # Fallbacks used when a hosted integration provides server-side names only.
8 | SUPABASE_URL=
9 | SUPABASE_PUBLISHABLE_KEY=
10 | SUPABASE_ANON_KEY=
11 | SUPABASE_SERVICE_ROLE_KEY=
12 | SUPABASE_SECRET_KEY=
13 | # Optional isolated Supabase project for Vercel Preview / test branches.
14 | # Used automatically when VERCEL_ENV=preview, or locally with
    SUPABASE_RUNTIME_TARGET=preview.
15 | SUPABASE_RUNTIME_TARGET=
16 | NEXT_PUBLIC_SUPABASE_PREVIEW_URL=
17 | NEXT_PUBLIC_SUPABASE_PREVIEW_PUBLISHABLE_KEY=
18 | SUPABASE_PREVIEW_URL=
19 | SUPABASE_PREVIEW_PUBLISHABLE_KEY=
20 | SUPABASE_PREVIEW_ANON_KEY=
21 | SUPABASE_PREVIEW_SERVICE_ROLE_KEY=
22 | SUPABASE_PREVIEW_SECRET_KEY=
23 | TSZR15_ADMIN_TOKEN=
```

Grupo	Variaveis	Papel
WhatsApp	NEXT_PUBLIC_WHATSAPP_BUSINESS_NUMBER, WHATSAPP_BUSINESS_NUMBER	Numero usado na URL wa.me.
Supabase publico	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Auth e leituras publicas/autenticadas.
Supabase servidor	SUPABASE_SERVICE_ROLE_KEY, SUPABASE_SECRET_KEY	Pedidos de convidado, admin, storage, cupons e rastreio.
Preview	NEXT_PUBLIC_SUPABASE_PREVIEW_URL, SUPABASE_PREVIEW_SERVICE_ROLE_KEY	Projeto isolado para preview/teste.
Admin	TSZR15_ADMIN_TOKEN	Senha/token do painel interno.
Site	NEXT_PUBLIC_SITE_URL, SITE_URL, VERCEL_URL	Origem segura para reset de senha e callbacks.

13. Tecnologias planejadas ou documentadas para evolucao

docs/APIS-NECESSARIAS.md lista integracoes para evoluir o MVP. Elas nao devem ser confundidas com features ja implementadas no codigo atual.

Tecnologia	Possivel uso	Status
Mercado Pago	Pagamento Pix/link/cartao e webhooks de confirmacao.	Planejado
WhatsApp Cloud API	Mensagens oficiais, webhooks e templates.	Planejado

Tecnologia	Possivel uso	Status
Shopee Open Platform	Automacao futura somente se o fluxo for permitido oficialmente.	Planejado
AliExpress/DSers	Fornecedor ou integrador homologado para compra assistida.	Planejado
Correios/Melhor Envio	Frete, prazo, etiquetas ou rastreio quando houver estoque proprio.	Opcional
Resend	Email transacional para comprovantes e atualizacoes.	Planejado
Nota fiscal/ERP	Explicitamente fora do escopo planejado atual.	Fora do escopo

docs/APIS-NECESSARIAS.md - ordem recomendada

```

286 | ## Ordem recomendada de implementacao
287 |
288 | 1. Banco de dados e pedido salvo.
289 | 2. Consentimento de compra assistida.
290 | 3. Painel admin para registrar compra em Shopee/AliExpress manualmente.
291 | 4. Mercado Pago para pagamento confirmado.
292 | 5. ViaCEP para reduzir erro de endereco.
293 | 6. Rastreio manual e notificacao por WhatsApp.
294 | 7. Automacao de Shopee/AliExpress somente depois de validar acesso oficial ou
    | integrador homologado.

```

14. Roteiro de estudo do projeto

- Comece em README.md para entender produto, rotas e variaveis.
- Leia package.json para ver stack e scripts executaveis.
- Abra app/layout.js, app/page.js e app/api/checkout/whatsapp/route.js para entender o App Router.
- Siga src/catalog e src/components/catalog para entender vitrine, busca, produto e carrinho.
- Siga src/checkout para entender totais, cupom, validacao e mensagem WhatsApp.
- Leia src/lib/supabase e supabase/migrations para entender dados, Auth, RLS, RPC e Storage.
- Leia src/admin e app/admin/actions.js para entender o backoffice e revalidacao.
- Rode npm run validate e npm test quando as dependencias estiverem instaladas corretamente.

Checklist mental

Pergunte sempre: este dado vem do cliente ou do servidor? Ele pode ir para o browser? Ele depende de service_role? A resposta define se o codigo deve ficar em componente cliente, Server Action, API route, proxy ou RPC SQL.